TRABAJO PRÁCTICO INTEGRADOR

Búsqueda de funciones similares por AST

UNIVERSIDAD CATÓLICA DE SANTIAGO DEL ESTERO

Lenguajes Formales y Autómatas — Ingeniería en Informática

1. Diseño del proceso

Extracción directa de función objetivo y de otras funciones del directorio → Normalización → Tokens → Similitud por Jaccard de n-gramas → Ranking top-K *(armado a medida que se van encontrando funciones)*.

1. **Extracción:** *rglob* sobre los ***.py***, *ast.parse* por archivo, y se toman *FunctionDef* y *AsyncFunctionDef*. Usamos una representación de *Función* más *"high-level"* que encapsula lo necesario.
2. **Normalización**: Se hace *deepcopy* del subárbol (no se modifica el original) y se recorre en ***preorden***. En el mismo recorrido se normaliza cada nodo y se emite su token: el tipo de nodo (*Assign*, *BinOp*, …) y su valor normalizado.
3. **Similitud**: Jaccard sobre 3-gramas de tokens.
4. **Ranking**:
   - Comparación todos-contra-uno, **O(n).** Es *candidata a duplicado* si score ≥ 70%
   - Ranking: **O(n log K)**, lo acotamos a K porque conocemos el límite y si no usamos uno por defecto, usamos búsqueda binaria por si se necesita un ranking grande.

2. Decisiones de Normalización

Qué	Cómo	Por qué
Nombre de la función	_	No debe influir (por consigna)
Variables y parámetros	VAR1, VAR2, … por orden de aparición, un mapa por función	Preserva la correspondencia: a + a ≠ a + b
Literales	LIT	Mismo cálculo/lógica con otras constantes es duplicado.
Docstring	Se elimina	Es documentación y tampoco debe influir en la comparación.

Qué	Cómo	Por qué
Builtins (len, range, …)	Se dejan tal cual, solo se normalizan al asignarse como variable	**len(VARn)** -> cuenta como builtin. **len = 1** -> se redefine y pasa a ser *VARn*.
Keywords	No se normalizan	Solo se reescriben Name, arg y Constant.

NOTA: No se normalizan operadores ni estructuras, un if sigue siendo If. Así, **sumar(x, y): return x + y** y **agregar(a, b): return a + b** generan **la misma** secuencia, y las lógicas quedan distintas.

3. Métrica Usada: Jaccard de n-gramas

Se arman los conjuntos de n-gramas (*n* tokens consecutivos) de cada secuencia: Es barato (intersección de conjuntos, cacheables por función), no requiere alinear las secuencias y, al trabajar con conjuntos de n-gramas, tolera el reordenamiento. Levenshtein, en cambio, penaliza mucho mover bloques (líneas).

Por qué n = 3. Con **n = 1** se ignora el orden y con n grande cualquier cambio chico rompe varios n-gramas. Se probó n = 1, 2, 3 y 4 sobre algunos tests:

n	Clon renombrado	Clon reordenado	Distintas (sumar/máximo)	Distintas (factorial/parsear enteros)
1	1,00	1,00	0,64	0,70
2	1,00	1,00	0,45	0,48
3	**1,00**	**1,00**	**0,27**	**0,21**
4	1,00	0,83	0,13	0,10

n = 3 es el valor que mejor separa clones de no clones: mantiene los clones en 1,00 y baja las distintas a 0,27 y 0,21. Con n = 4 el clon reordenado ya baja a 0,83, pero se que es más restrictivo. Con n = 1 el segundo negativo queda en 0,70, que alcanza el umbral (≥ 0,7) y se marcaría como duplicado: un falso positivo.

Umbral 0,7. Se fijó *después* de elegir n. Cualquier valor entre 0,27 (máx. no-clon) y 1,00 (mín. clon) separa los duplicados, se eligió 0,7 por seguridad, con un margen a ambos lados.

4. Limitaciones conocidas

- **Clones de ciclos:** for y while, o recursión e iteración, tienen otra forma de AST y tienen baja similitud aunque hagan lo mismo.
- **Funciones muy cortas:** pocos n-gramas, scores no muy precisos y posibles falsos positivos.
- **Conjuntos:** Jaccard no cuenta repeticiones ni el orden total más allá de cada n-grama, dos funciones con los mismos n-gramas en otro orden pueden dar score alto aunque hagan cosas distintas.
- **Keywords / except as:** no se tokenizan, así que x.append(y) y x.remove(y) producen la misma secuencia.

5. Posibles mejoras

- **Normalizar construcciones equivalentes** *(for/while, if/else invertido, x += 1 vs x = x + 1).*
- **Tokenizar y Normalizar los Keywords / except as.**
- **Métrica combinada** Jaccard + Levenshtein, y umbral mínimo de tamaño para ignorar funciones triviales.
- **Tokenizar nombres de atributos y keywords**, para distinguir append de remove.